5.3 Class model

Project: Electrical Consumption Monitor — EnergyMonitor **Record:** 3145556 — Software Analysis and Development (ADSO), SENA CIES Regional Huila **Version:** 1.0

> **Update note (10/27/2026).** This document describes the design of version 1.0, when the backend was the `energy-monitor-backend` modular monolith (Spring Modulith). That design evolved: the seven bounded contexts are now **seven microservices** behind the `en-monitor-api-gateway` API gateway (8080) — security 8082, device 8084, measurement 8085, home 8086, notification 8087, alert 8088 and recommendation 8089 —, communicating through internal REST (`/internal/**`) and RabbitMQ events with a transactional outbox, on the same shared MySQL schema (37 tables). The modular backend was retired. The hexagonal layers, contexts and patterns described here live on inside each service; the current architecture, routes, events and configuration are in the technical manual (sections 9 to 13) and the current data model in `06-base-datos`.

1. Purpose

The class model describes the internal structure of the system: the classes, interfaces, enumerations and relationships that make up the domain. Unlike the data model, which answers "how the data is stored", the class model answers "how the system behaves": what responsibilities each class has, what methods it exposes and how they collaborate with each other.

This model is the conceptual basis of the design patterns described in `05.4-design-patterns.md`.

2. Class diagram

The base diagram is located at `assets/diagramas/Diagrama_clases.wsd` (PlantUML syntax, renderable at https://www.plantuml.com/plantuml/uml).

Figure 2. *Class diagram of the domain, organized into packages: Strategy, enumerations, domains and Observer / Factory Method patterns.*

> **Version note.** The attached diagram was produced in an earlier phase of the project. Several classes that are marked there as "pending implementation" already exist today in the code, and the enumerations have grown. Section 10 documents the difference precisely.

3. Packages of the model

Package	Content	Responsibility
`Strategy_Pattern`	`UserPermissions`, `MemberPermissions`, `OwnerPermissions`	Authorization according to the role within the home
`enums`	Domain and API enumerations	Sets of allowed values
`Domain_Security`	`Person`, `User`, `UserConfiguration`, `PasswordResetToken`, `UserSession`, `RefreshToken`, `SecurityConfiguration`, `PasswordPolicy`, `LoginErrorLog`, `Email`, `PasswordHash`	Identity, credentials and sessions
`Domain_Access_Control`	`SystemRole`, `UserSystemRole`, `Permission`, `SystemRolePermission`, `AuditLog`, `CheckPermission`	RBAC and auditing
`Domain_Home_Management`	`HomeType`, `HomeThresholds`, `Home`, `UserHome`, `Role`	Homes, members and thresholds
`Domain_Devices`	`ApplianceType`, `Device`, `DeviceStatusLog`, `DeviceHome`, `DeviceStatus`	Measuring devices

Package	Content	Responsibility
`Domain_Measurements`	`Measurement`, `ConsumptionLevel`	Measurements and risk levels
`Domain_Notifications`	`Alert`, `Recommendation`, `Notification`, `NotificationPreference`, `PushSubscription`	Events, deliveries and preferences
`Observer_Pattern`	`MeasurementObserver`, `AlertObserver`, `RecommendationObserver`	Reaction to measurements
`Factory_Method_Pattern`	`AlertFactory`, `RecommendationFactory`, `NotificationFactory`	Construction of events and notices
`Recommendation`	`RecommendationRules`, `Suggestion`, `MonitoredDevice`, `HourSample`, `ConsumptionLimit`	Domain service of the consumption analysis

4. UML notation used

Symbol	Meaning
`+`	Public method
`-`	Private attribute
`{abstract}`	Abstract class or method
`<<interface>>`	Interface
`<<FK>>`	Attribute that references another entity, without navigable association
`..>`	Dependency (usage)
`--`	Association
`*--`	Composition (the whole controls the lifecycle of the part)
`o--`	Aggregation

> **Convention of this project:** the relationships "drawn" toward tables of other contexts are represented as **dotted dependency** (`..>`), not as association. It is the graphical consequence of the flat relational mapping decision described in `05.2-data-model.md`, section 4.4.

5. Classes of the security domain

5.1 `Person`

Human person associated with an account. Deliberately **does not** store contact data or an address.

Attribute	Type	Visibility
`idPerson`	`String`	`-`
`name`	`String`	`-`
`lastName`	`String`	`-`

Operation	Responsibility
`+ rename(name, lastName)`	Changes first and last name, validating that they are not left empty

Design decisions documented in the model and in the code:

Discarded attribute	Reason	Changeset that removed it
`person.address`	The authoritative address is `home.address`; duplicating it would generate inconsistencies	`security-018`
`person.cellphone`	The account is identified exclusively by email address	`security-020`

Discarded attribute	Reason	Changeset that removed it
`person.profileImage`	The avatar belongs to the account, not to the person; it lives in `User`	`security-017`

5.2 `User`

Access account. It concentrates credentials, status and session traceability.

Attribute	Type	Visibility
`idUser`	`String`	`-`
`personId`	`String`	`-`
`passwordHash`	`PasswordHash`	`-`
`email`	`Email`	`-`
`emailVerified`	`boolean`	`-`
`registrationDate`	`Instant`	`-`
`status`	`UserStatus`	`-`
`failedLoginAttempts`	`int`	`-`
`lastLoginAt`	`Instant`	`-`
`profileImage`	`String`	`-`

Operation	Responsibility
`+ canAuthenticate()`	Indicates whether the account is `ACTIVE` and the email is verified
`+ changePassword(newHash)`	Replaces the hash and revokes the active sessions
`+ changeEmail(newEmail)`	Changes the email and forces re-verification
`+ changeProfileImage(profileImage)`	Updates the image (data URL)
`+ verifyEmail()`	Marks the email as verified
`+ activate()` / `+ deactivate()` / `+ block()`	State transitions
`+ incrementFailedLoginAttempts()`	Counts failed attempts
`+ resetFailedLoginAttempts()`	Resets the counter after a successful sign-in
`+ recordSuccessfulLogin(instant)`	Records `lastLoginAt`

> **Applied pattern — Domain Object (Entity).** The behavior of `User` is not delegated to an external service: the state transition rules (`ACTIVE → BLOCKED`) and the invariant control live inside the class. This keeps the aggregate invariant guaranteed without depending on every caller remembering to validate it.

5.3 Value objects: `Email` and `PasswordHash`

Both are **immutable records** with a static validating factory (`Email.of(...)`, `PasswordHash.of(...)`) and an accessor without the `get` prefix (`user.email()`).

Property	Design consequence
Invariants at construction time	An `Email` instance always contains a validly formatted email address
Equality by value	Two `Email` with the same content are equal; the `Set` of emails works
No identity	They are not entities; they have no primary key nor their own persistence

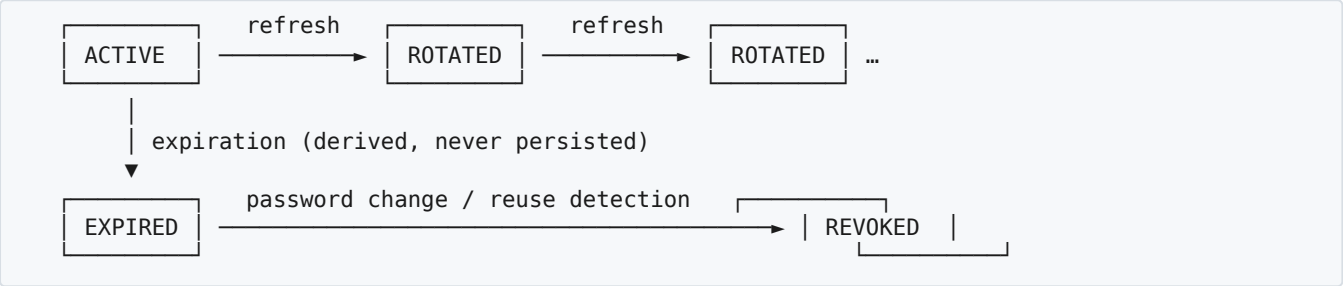
> **Applied pattern — Value Object.** It is the piece that prevents the domain from handling loose strings for emails and passwords, and that concentrates the format rules in a single place.

5.4 `UserSession` and `RefreshToken`

The pair of classes that implements the session lifecycle with token rotation.

Class	Key operations
`UserSession`	`rotate(newExpiration, rotatedAt)`, `revoke(revokedAt, reason)`, `close(instant)`, `isExpired(now)`, `isActive(now)`
`RefreshToken`	`RefreshToken.root(...)`, `RefreshToken.childOf(...)`, `isExpired(now)`, `isUsable(now)`, `isReplayed()`, `rotate(rotatedAt)`, `revoke(revokedAt, reason)`

`RefreshTokenStatus` state machine:



Composition `UserSession 1 *-- 1..* RefreshToken`**.** A session owns all the refresh token generations generated under it. When the session is revoked, all of them are invalidated: this is the implementation of the global session close.

Self-relation `RefreshToken 0..1 -- 1 RefreshToken : parent`**.** One generation points to the one it replaces, forming a genealogical chain that allows detecting the reuse of an already rotated token.

Declared technical debt in the code: the token family has no size limit; an attacker who raids a valid `refresh_token` can accumulate generations until the anomaly is detected.

5.5 `PasswordPolicy` and `CheckPermission`

Class	Operation	Rule
`PasswordPolicy`	`+ validate(password)`	Length 8–64, requires uppercase, numbers and symbols, 90-day validity
`CheckPermission`	`+ check(idUser, permissionCode)`	Walks the five-table RBAC chain against live state

> `CheckPermission` is the **only real authorization path** of the system. It contains no logic per role type and does not interpret permissions embedded in the JWT: it always resolves against the database, so that a role change takes effect immediately without waiting for the token to expire.

6. Classes of the homes domain

Class	Operations
`HomeType`	Catalog of home types
`HomeThresholds`	`+ resetToDefaults()`
`Home`	`+ addDevice(d)`, `+ removeDevice(d)`, `+ addUser(u, role)`, `+ removeUser(u)`, `+ updateThresholds(daily, monthly)`, `+ getAllDevices()`
`UserHome`	`+ toggleFavorite()`, `+ leaveHome()`, `+ getPermissions()`

Design decisions:

Decision	Justification
`Home 1 *-- 1 HomeThresholds` (composition)	Thresholds have no existence of their own without a home. If the home is deleted, its thresholds are deleted with it.
`Home "*" o-- "1" HomeType` (aggregation)	The home type is a shared catalog: the same `HomeType` serves many homes and does not "belong" to any of them.
`Home 1 -- "0..*" UserHome` (association)	Membership in the home exists on its own, even if the home is deleted.
`+ getPermissions()` in `UserHome`	Returns the permissions object (`UserPermissions`) corresponding to the role. Extension point of the Strategy pattern.

7. Classes of the devices and measurements domain

Class	Operations
`Device`	`Device.create(...)`, `Device.relinked(...)`, `Device.edited(...)`, `+ withApiKey(newKey)`, `+ takeMeasurement()`
`DeviceStatusLog`	`DeviceStatusLog.create(...)` — status, signal strength and last connection
`Measurement`	`+ getVoltage()`, `+ getCurrent()`, `+ getActivePower()`, `+ getStoredEnergy()`
`ConsumptionLevel`	`minLimit – maxLimit` range of the risk level

> **Applied pattern — Subject / Observer.** The class diagram models `MeasurementObserver`, `AlertObserver` and `RecommendationObserver` as observers of `Device`. In the implementation, the subject role is played by the **event publisher** of the `measurement` module, and the observer role is played by the `MeasurementRecordedListener` listener of the `alert` module. The contract is preserved; the transport changes from direct invocation to an in-process event (see `05.4-design-patterns.md`, section 4).
>
> **Applied pattern — Value Object.** `Measurement` has no identity of its own beyond `idMeasurement` and exposes its magnitudes through getters. The electrical magnitudes are immutable values: they cannot change once the measurement is recorded, which guarantees the reliability of the history.

8. Event and notice classes

Class	Operations
`Alert`	Factories `Alert.threshold(...)`, `Alert.limit(...)`, `Alert.connectivity(...)`, `Alert.device(...)`; `+ resolveAlert()`
`Recommendation`	`+ markAsRead()`, `+ markAsUnread()`
`Notification`	`+ settled(...)` (`SENT`/`FAILED`), `+ read(...)`
`NotificationPreference`	`NotificationPreference.defaults()`
`RecommendationRules`	`+ evaluate(instant, zoneId, devices, samples, limit)`

Central design decision: `Notification` is not the base class of `Alert` and `Recommendation`.

In previous versions of the model it was usual to model `Notification` as the superclass of the events. The definitive model separates them because they answer different questions:

Class	Question it answers	Context
`Alert` / `Recommendation`	**What happened?**	Alerts / Recommendations
`Notification`	**Did the message arrive, through which channel and was it read?**	Notifications

`Notification` references the source event through the `sourceType` + `sourceId` pair, without coupling to its schema. In addition, its state cycles (`deliveryStatus` and `readAt`) are **independent of each other and independent of** `alert.alertStatus`:

> A delivered alert is not a resolved alert.

Immutable class by exception. `RecommendationEntity` is the **only** persistence entity in the project without setters: it declares only its own `@PrePersist` and `@PreUpdate`, and only allows the read status to change. This reinforces the invariant that recommendations are immutable once created.

8.1 `RecommendationRules` — domain service

It is the **only** class in `domain/service` in the project. It is purely functional: it receives no dependencies, performs no input/output and does not read the clock (the instant and the time zone are passed as parameters).

Design decision	Justification
No dependencies	The same set of rules can be tested exhaustively without a Spring container
Clock and time zone as input	The result is deterministic and reproducible; it facilitates consumption rule tests
Lives in the domain, not in the application	Business rules do not depend on the framework

> **Applied pattern — Domain Service.** When a behavior spans several aggregates and does not naturally belong to any of them, it is extracted into a domain service. The recommendation rules analyze measurements from **all** the devices of a home, so it does not correspond to `Measurement`, to `Device` nor to `Recommendation` in particular.

9. Domain enumerations

Enumeration	Values	Location	Use
`UserStatus`	`ACTIVE`, `INACTIVE`, `BLOCKED`	Domain	Account status
`AuditAction`	`CREATE`, `UPDATE`, `DELETE`, `LOGIN`, `LOGIN_FAILED`, `LOGOUT`	Domain	Audited action
`LoginErrorType`	`INVALID_PASSWORD`, `USER_NOT_FOUND`, `ACCOUNT_BLOCKED`, `ACCOUNT_INACTIVE`	Domain	Classification of the access failure
`RefreshTokenStatus`	`ACTIVE`, `ROTATED`, `REVOKED`, `EXPIRED`	Domain	Token cycle (`EXPIRED` is derived, not persisted)
`RevocationReason`	`REFRESH_TOKEN_REUSE`, `PASSWORD_CHANGED`, `ADMINISTRATIVE`	Domain	Revocation reason
`Role`	`OWNER`, `MEMBER`	Domain	Role of the user within the home
`LimitPeriod`	`DAILY`, `MONTHLY`	API	Period of the threshold being edited
`DeviceStatus`	`ONLINE`, `OFFLINE`	Domain	Device connectivity
`ConnectivityStatus`	`ONLINE`, `OFFLINE`	API	Connectivity exposed to the client
`RiskConsumption`	`LOW`, `MEDIUM`, `HIGH`, `CRITICAL`	API	Consumption risk level
`AlertType`	`THRESHOLD`, `LIMIT`, `CONNECTIVITY`, `DEVICE`	API	Origin of the alert
`AlertStatus`	`PENDING`, `RESOLVED`	API	Lifecycle of the alert
`RecommendationType`	`SAVING`, `PEAK_HOURS`, `HIGH_CONSUMPTION`, `HISTORICAL_COMPARISON`, `THRESHOLD`	API	Reason for the recommendation
`RecommendationStatus`	`READ`, `UNREAD`	API	Read state of the recommendation

Enumeration	Values	Location	Use
`NotificationStatus`	`PENDING`, `SENT`, `FAILED`	Domain	Result of the delivery
`NotificationChannel`	`EMAIL`, `PUSH`	Domain	Delivery channel
`NotificationSourceType`	`ALERT`, `RECOMMENDATION`, `SECURITY`	Domain	Context that originated the message
`Access` (in `AuthorizeBrokerAccess`)	`READ`, `WRITE`, `READ+WRITE`, `SUBSCRIBE`	Domain	Access permissions to the MQTT broker

> **All enumerations are declared in Java**, unlike the MySQL `ENUM` columns that share the name. This allows the domain and the database to share vocabulary without coupling the business logic to the storage engine. Those in the "API" column live in the `api/` package of the module because they are part of the public contract.

10. Differences between the design artifact and the implemented model

#	Aspect	Diagram of the artifact	Current code	Origin of the change
1	`UserPermissions`, `MemberPermissions`, `OwnerPermissions`	Marked as *"pending implementation"*	**Implemented** in `home/domain/model`; `UserHome.getPermissions()` returns the corresponding strategy	Requirement RF-HOM-05 (permission management)
2	`RecommendationAnalyzer`	Class with `analyze()`, `detectPatterns()`, `generateRecommendation()`	Replaced by `RecommendationRules` (domain service) + `EvaluateRecommendationsService` (use case) + `RecommendationFactory`	Separation between business rule and use case
3	`AlertType`	`THRESHOLD`, `CONNECTIVITY`	+ `LIMIT`, `DEVICE`	Accumulated limit alerts and linking alerts
4	`Notification` without subclasses	Without `NotificationPreference` nor `PushSubscription`	Both classes added to `Domain_Notifications`	Channel preferences and Web Push
5	`Device.subscribe/unsubscribe/notify`	Subject methods	Replaced by Spring event publishing and listening (`ApplicationEventPublisher` + `@EventListener`)	Decoupling between contexts
6	`Notification.markAsSent/markAsFailed/markAsRead`	Three methods	A single `settled(...)` operation plus `read(...)`	Simplification of the state transition
7	`SystemConfig`	Mentioned in a comment about `useSystemDefault`	**Removed**; the default value lives in `HomeProperties` (configuration, not database)	Closing of the design gap
8	`Recommendation` as an event of its context	Identical	It is maintained; `status` is materialized in `NotificationsStatus`	No changes

Recommendation: update `Diagrama_clases.wsd` with these eight differences before the final delivery, especially the first one, which today makes it seem that authorization by role does not exist.

11. Design patterns identified in the class model

Diagram package	Status	Concrete implementation
`Strategy_Pattern`	**Implemented**	`UserPermissions` with two concrete strategies per role
`Observer_Pattern`	**Implemented with event transport**	`MeasurementRecorded` published by `measurement` and listened to by `alert`

Diagram package	Status	Concrete implementation
`Factory_Method_Pattern`	**Implemented**	Static factories in the aggregates: `Alert.threshold()`, `RefreshToken.root()`, `Device.create()`
Value object	**Implemented**	`Email`, `PasswordHash`, `Measurement`
Domain object	**Implemented**	Behavior in `User`, `RefreshToken`, `UserSession`
Domain service	**Implemented**	`RecommendationRules`
Singleton	**Implemented by the container**	Spring beans, `Clock`, `ThreadPoolExecutor`

The complete detail of each pattern, with the comparison between the canonical pattern and its real application, is developed in `05.4-design-patterns.md`.

12. Class → requirement traceability matrix

Class or package	Requirement (requirements specification)
`Person`, `User`, `UserConfiguration`	RF-SEC-01, RF-SEC-08
`Email`, `PasswordHash`, `PasswordPolicy`	RF-SEC-12
`PasswordResetToken`, `LoginErrorLog`	RF-SEC-06; RNF-SEG-05, RNF-SEG-06
`UserSession`, `RefreshToken`	RF-SEC-04, RNF-SEG-01, RNF-SEG-02 (persistent session)
`SystemRole`, `UserSystemRole`, `Permission`, `SystemRolePermission`, `CheckPermission`	RF-SEC-09 (permission management)
`AuditLog`	RF-SEC-12; RNF-SEG (information security)
`Home`, `HomeType`, `UserHome`	RF-HOM-01 to RF-HOM-04
`HomeThresholds`	RF-HOM-08
`UserPermissions`, `MemberPermissions`, `OwnerPermissions`	RF-HOM-05 (authorization by role)
`Device`, `ApplianceType`, `DeviceStatusLog`	RF-DEV-02 to RF-DEV-06; RF-MED-04 to RF-MED-06
`Measurement`, `ConsumptionLevel`	RF-MED-01, RF-MED-02, RF-MED-09
`Alert`	RF-ALE-01 to RF-ALE-06
`Recommendation`, `RecommendationRules`	RF-REC-01 to RF-REC-09
`Notification`, `NotificationPreference`, `PushSubscription`	RF-NOT-01, RF-NOT-02, RF-NOT-04 (delivery channel); RF-NOT-03

13. Related artifacts

Artifact	Location
Class diagram (PlantUML)	`assets/diagramas/Diagrama_clases.wsd`
Data model	`05.2-data-model.md`
Design patterns	`05.4-design-patterns.md`